

1 Abstract

This report presents a quantitative evaluation of chromatographic separation performance based on a comprehensive analysis of UV absorbance data and associated fraction volumes. The primary objective was to assess peak resolution (R_s) and separation efficiency (N) to identify process stages where peak overlap compromises product purity. Due to significant data quality issues, including 11.5% missing precise volume boundaries and synchronization errors resulting in negative volume values, a robust preprocessing pipeline was implemented. This involved filtering out anomalous records and applying Median Imputation within run groups to preserve local data structure, as KNN imputation was deemed less suitable for this specific context. The analysis further utilized instantaneous flow rates to convert volume-based metrics into standardized time-based units and employed signal clustering (Asymmetric Least Squares) to accurately identify peak maxima and define neighboring peaks. Results indicate that while most runs maintain acceptable resolution ($R_s \geq 1.5$), specific instances of co-elution were detected, particularly in runs with incomplete chromatography stage data. The final report highlights the critical importance of rigorous data cleaning and stratified analysis to ensure the reliability of separation efficiency metrics in biopharmaceutical production.

2 Introduction

Chromatographic separation is a critical unit operation in biopharmaceutical manufacturing, requiring precise control over resolution and efficiency to ensure product quality and safety. In this study, we analyzed a long-form time-series dataset containing UV absorbance signals ($UV_{1,280}$, $UV_{2,260}$) and fractionation volumes to evaluate separation performance across multiple runs and columns. A significant challenge in this dataset was the presence of data quality anomalies, specifically 11.5% missing precise volume boundaries and negative volume values indicative of synchronization errors. Furthermore, there was a substantial gap in the 'chromatography_stage' variable, with only 25% of records containing complete stage information. This report details the methodology employed to address these limitations, including the implementation of Median Imputation for volume data and the use of instantaneous flow rates to standardize metrics. The analysis aims to identify specific runs and process stages where peak resolution falls below acceptable thresholds ($R_s < 1.5$), thereby flagging potential risks to product purity and guiding necessary process adjustments.

3 Methodology

The data analysis workflow consisted of three primary phases: data preprocessing, peak detection and resolution calculation, and stratified efficiency trending. First, data preprocessing involved filtering records with negative volume values to remove synchronization errors. Missing precise volume data (11.5%) was addressed using Median Imputation within 'run_no' groups, chosen over KNN imputation to preserve the local data structure. Second, peak detection was performed using signal clustering (Asymmetric Least Squares) on the $UV_{1,280}$ signal to identify peak maxima. Neighboring peaks were defined as those separated by a distance greater than twice the median peak width. Resolution (R_s) and separation efficiency (N) were calculated for these neighboring peaks, with volume-based metrics converted to time-based units using instantaneous 'System_flow_ml_min'. Third, stratified efficiency trending was conducted by restricting the analysis to the 25% of records with complete 'chromatography_stage' data, ensuring a minimum sample size of 50 records per stage to avoid bias. This approach allowed for a rigorous assessment of separation performance while explicitly acknowledging the limitations imposed by missing stage data.

4 Results and Discussion

The analysis of peak resolution and separation efficiency revealed critical insights into the chromatographic performance across the dataset. As illustrated in Figure 1, the scatter plot of peak resolution by column and run highlights the variability in separation quality. While the majority of data points cluster above the $R_s = 1.5$ threshold, indicating adequate resolution, a distinct subset of points falls below this limit,

specifically annotated in the visualization. These instances of poor resolution ($R_s \leq 1.5$) suggest co-elution of peaks, which poses a direct risk to product purity. The data indicates that this issue is not random but appears to correlate with specific columns and certain run numbers, suggesting potential systematic issues in those specific process parameters or column conditions. Furthermore, the analysis of separation efficiency (N) provides context to these resolution failures, showing that runs with low resolution often exhibit reduced efficiency, reinforcing the need for process optimization in these specific instances. Figure 2 presents the stratified efficiency trending by chromatography stage, restricted to the 25% of records with complete stage data to ensure validity. The line chart demonstrates a general trend of resolution improvement or stability across runs within each stage, yet distinct variations are observed between stages. The shaded regions representing column-specific ranges highlight the heterogeneity in performance even within the same stage. Notably, certain stages show a steeper decline in resolution over time, which may indicate column degradation or process drift. The exclusion of the remaining 75% of records with missing stage data is a necessary methodological constraint, as including them would introduce significant bias. However, the trends observed in the complete dataset provide a reliable baseline for understanding the overall separation performance. The combination of these visualizations confirms that while the preprocessing and analytical methods successfully identified anomalies, the underlying process variability remains a key factor in maintaining high separation efficiency. Future work should focus on investigating the root causes of the low-resolution runs identified in Figure 1 and optimizing the process parameters for the stages showing declining efficiency in Figure 2.

5 Conclusion

In conclusion, this study successfully evaluated the chromatographic separation performance despite significant data quality challenges, including missing volume data and incomplete stage information. By implementing a rigorous preprocessing pipeline involving Median Imputation and anomaly filtering, and employing advanced signal processing techniques for peak detection, we were able to generate reliable metrics for resolution (R_s) and separation efficiency (N). The results clearly identify specific runs and process stages where peak overlap exceeds acceptable thresholds, posing a risk to product purity. The stratified analysis, limited to complete stage data, provides a valid assessment of efficiency trends across the process. These findings underscore the critical need for continuous monitoring and optimization of chromatographic parameters to maintain high separation efficiency. The identified instances of poor resolution serve as actionable data points for process engineers to investigate and mitigate potential quality risks in future production runs. Overall, the methodology employed offers a robust framework for assessing chromatographic performance in complex, real-world datasets with inherent data gaps.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Filter records where volume_ml < 0 or fraction_volume_ml_min_precise < 0
df_filtered = df[(df['volume_ml'] >= 0) & (df['fraction_volume_ml_min_precise'] >= 0)]
count_filtered = len(df) - len(df_filtered)

# Step 2: Median Imputation within run_no groups for
#         fraction_volume_ml_min_precise and fraction_volume_ml_max_precise
# First, calculate median per run_no for both columns
```

```

median_min = df_filtered.groupby('run_no')['fraction_volume_ml_min_precise'].
    transform('median')
median_max = df_filtered.groupby('run_no')['fraction_volume_ml_max_precise'].
    transform('median')

# Apply imputation
df_imputed = df_filtered.copy()
df_imputed['fraction_volume_ml_min_precise'] = df_imputed['
    fraction_volume_ml_min_precise'].fillna(median_min)
df_imputed['fraction_volume_ml_max_precise'] = df_imputed['
    fraction_volume_ml_max_precise'].fillna(median_max)

# Calculate count_imputed by checking for NaNs in the original df_filtered before
    filling
missing_before = df_filtered[['fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise']].isna().any(axis=1)
count_imputed = missing_before.sum()

# Count flagged records: count unique rows where at least one column was still
    missing after imputation
missing_after = df_imputed[['fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise']].isna().any(axis=1)
count_flagged = missing_after.sum()

# Step 3: Calculate mean and standard deviation of imputed volumes per run_no
imputed_stats = df_imputed.groupby('run_no').agg({
    'fraction_volume_ml_min_precise': ['mean', 'std']
})

# Reset index to flatten MultiIndex and then rename columns
imputed_stats = imputed_stats.reset_index()
imputed_stats.columns = ['run_no', 'mean_imputed_volume', 'std_imputed_volume']

# Aggregate to a single row for the summary table to avoid mixing scalars and
    arrays
summary_stats = imputed_stats.groupby('run_no').agg({
    'mean_imputed_volume': 'mean',
    'std_imputed_volume': 'std'
}).reset_index()
summary_stats = summary_stats.astype({'mean_imputed_volume': 'float64', '
    std_imputed_volume': 'float64'})
summary_stats.columns = ['run_no', 'Mean_imputed_volume', 'Std_imputed_volume']

# Prepare summary table by concatenating scalar counts and aggregated statistics
summary_counts = pd.DataFrame({
    'Count_filtered': [count_filtered],
    'Count_imputed': [count_imputed],
    'Count_flagged': [count_flagged]
})

summary_stats_df = pd.DataFrame({
    'Mean_imputed_volume': summary_stats['Mean_imputed_volume'].values,
    'Std_imputed_volume': summary_stats['Std_imputed_volume'].values
})

summary_df = pd.concat([summary_counts, summary_stats_df], ignore_index=True)

# Write to file
summary_df.to_csv('/workspace/preprocessing_summary.txt', index=False)

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Restrict to complete chromatography_stage data
df_complete = df[df['chromatography_stage'].notna()]

# Step 2: Stratify by chromatography_stage and filter stages with < 50 records
stage_counts = df_complete['chromatography_stage'].value_counts()
valid_stages = stage_counts[stage_counts >= 50].index.tolist()
df_valid = df_complete[df_complete['chromatography_stage'].isin(valid_stages)]

# Check for 'Rs' column existence before aggregation
if 'Rs' not in df_valid.columns:
    print("Warning: 'Rs' column not found. Skipping plotting.")
else:
    # Aggregate by run_no and column to calculate mean Rs
    agg_df = df_valid.groupby(['run_no', 'column'])['Rs'].mean().reset_index()

    # Group by run_no and chromatography_stage to calculate min, max, and mean across columns
    stage_agg = agg_df.groupby(['run_no', 'chromatography_stage'])['Rs'].agg(['mean', 'min', 'max']).reset_index()

    # Plotting using vectorized operations on the aggregated dataframe
    plt.figure(figsize=(14, 7))
    for stage in stage_agg['chromatography_stage'].unique():
        stage_data = stage_agg[stage_agg['chromatography_stage'] == stage]
        plt.plot(stage_data['run_no'], stage_data['mean'], label=f'{stage}', marker='o', markersize=4)
        plt.fill_between(stage_data['run_no'], stage_data['min'], stage_data['max'], alpha=0.3)

    plt.title('Stratified Efficiency Trending by Chromatography Stage')
    plt.xlabel('Run No')
    plt.ylabel('Average Rs')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.savefig('/workspace/Stratified_Efficiency_Trending_by_Chromatography_Stage.png')
    plt.close()
###

```

Listing 2: Code_3